Real-world (a cold interview on your own project) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `project_walkthrough.md`: explaining your own project to someone who has never seen it is asked in nearly every AI engineering interview. The four beats — problem, shape, one decision, one limitation — give that explanation a structure you can reuse for any project. **If not:** you'd list features, and the interviewer would never hear *why* you built it that way.

Write your own answer first, in `practice/project_walkthrough.md`, then compare.

Model answer for: "Interview me about Project 4."

Solution

Story: the same four beats with the real mechanism and trade-off behind each one — the decision *and* what it cost. **If not:** the design would sound picked, not weighed.

"ContentForge solves content production for teams that need researched, reviewed briefs without one person running every step by hand.

It's Doc11's supervisor pattern with five agents: a supervisor that owns routing, a research agent that gathers findings, an analysis agent that works out what they mean, a writer that drafts from the analysis, and a reviewer that accepts or rejects the draft. Each agent writes its output into one shared state, and the supervisor reads that state after every step to pick the next agent — using `Command` for each hand-off.

The decision worth calling out: a supervisor instead of a fixed sequence, because review isn't a one-way gate — a rejected draft has to go *back* to the writer with the reviewer's feedback, and a fixed pipeline has no clean way to do that. The loop is bounded: after 3 rejections the run ends with a clear "couldn't agree" report and the full feedback history, instead of cycling forever. The trade-off I accepted: you can't read the code top to bottom and know the order anymore — you trace the supervisor's rules and the routing log, which records *why* each hop happened.

The honest limitation: the supervisor is rule-based, so every task goes through research first, even one that already includes the facts — a wasted, paid step. Given more time, I'd switch that one decision to Doc11's LLM-judged routing, and measure whether the saved research calls are worth the extra routing call."

Why this answer works: it's this document's Normal-depth rung, done properly — it opens with the user-facing problem, not the tech stack, which is what shapes an interviewer's first impression. It names the pattern *and* the specific requirement that pattern satisfies (backward routing when the reviewer rejects a draft), which is what "requirements → architecture is matching, not inspiration" (Doc16) actually looks like in speech. It states a real trade-off it accepted, not just a benefit, showing the choice was weighed, not just picked. And it volunteers the limitation with a concrete fix, unprompted.